Monitoramento AWS — Segurança de Identidades e Plano de Controle

Olá, meu nome é Cauã. Esta documentação apresenta os resultados dos meus estudos e a execução de uma atividade prática na plataforma TryHackMe, focada especificamente na segurança e no monitoramento da AWS.

📌 Introdução

Monitorar o plano de controle (*control plane*) é o primeiro passo para garantir a segurança de qualquer ambiente em nuvem. Dentro desse escopo, a autenticação é a área prioritária de foco: é essencial saber exatamente quem está fazendo login na AWS, de qual localização esse acesso se origina e quais credenciais estão sendo utilizadas.

Neste documento, você acompanhará a investigação dos ataques mais comuns contra identidades na AWS e as defesas utilizadas para mitigá-los. Ao longo da leitura, você verá:

- Os conceitos fundamentais de IAM (Gerenciamento de Identidade e Acesso), chaves de acesso, funções (*roles*) e políticas;
- Como o AWS CloudTrail registra os diferentes métodos de login na infraestrutura;
- A exploração de violações reais ocorridas em ambientes de nuvem e as estratégias para evitá-las;
- A aplicação prática da caça a ameaças (*Threat Hunting*), utilizando os logs para resolver desafios técnicos focados na detecção de invasões.

IAM e Credenciais de Usuário na AWS

O AWS Identity and Access Management (IAM) gerencia o acesso à nuvem através de quatro componentes essenciais:

- **Usuário (User):** Pessoa ou aplicação que acessa a AWS.
- **Grupo (Group):** Coleção de usuários.
- **Função (Role):** Identidade temporária assumida por usuários ou serviços.
- **Política (Policy):** Documento JSON que define as permissões.

Métodos de Autenticação

- **Console de Gerenciamento:** Acesso via navegador utilizando ID da conta, usuário, senha e MFA.
- **Chaves de Acesso (Access Keys):** Credenciais de longo prazo (ID e Segredo) para acesso programático (CLI/SDK). Fixá-las em código-fonte representa alto risco de vazamento e invasão.
- **Funções IAM (Roles) e Tokens STS:** A abordagem segura para autorizar serviços (como instâncias EC2). Em vez de chaves estáticas, a AWS gera credenciais temporárias válidas por poucas horas via STS (*Security Token Service*), eliminando a necessidade de fixar senhas no código e permitindo controles granulares de rede e acesso.

Bash
admin@it-srv:~$ aws configure # Configurando a chave de acesso (ID e Segredo)
admin@it-srv:~$ aws s3 ls thm-db-backups --recursive # Acesso programático autorizado aos recursos

📌 O que foi realizado

- **O que foi feito:** Mapeamento da estrutura de identidades da AWS e da transição entre métodos de autenticação estáticos e temporários.
- **Conceitos aprendidos:** A diferença crítica de arquitetura e segurança entre credenciais de longo prazo (Access Keys) e a assunção de papéis com credenciais de curto prazo (Tokens STS).
- **Importância:** Compreender a origem da autenticação é o pilar da defesa em nuvem (SOC). O vazamento de Access Keys estáticas é um vetor inicial massivo de invasões. Já o monitoramento de Tokens STS é essencial para rastrear movimentações laterais quando um atacante compromete um serviço (como o EC2) que possui uma Role vinculada.

Monitoramento de Logins no Console (AWS CloudTrail)

Embora ataques de força bruta contra o Console de Gerenciamento da AWS sejam raros devido ao MFA, campanhas de *phishing* são amplamente utilizadas para roubar credenciais e viabilizar ataques de *ransomware* na nuvem. O serviço AWS CloudTrail registra todas essas tentativas (com sucesso ou não) através do evento `ConsoleLogin`.

Campos Críticos para Investigação:

- `errorMessage`: Quando presente, indica que a autenticação falhou.
- `additionalEventData.MFAUsed`: Mostra se a autenticação multifator foi utilizada (o ideal é sempre "Yes").
- **Padrões Suspeitos:** Logins no usuário `root` (que deve ser usado apenas em emergências), usuários de serviço acessando via console (quando deveriam usar chaves programáticas) e acessos via IPs anonimizadores (Tor/VPN).

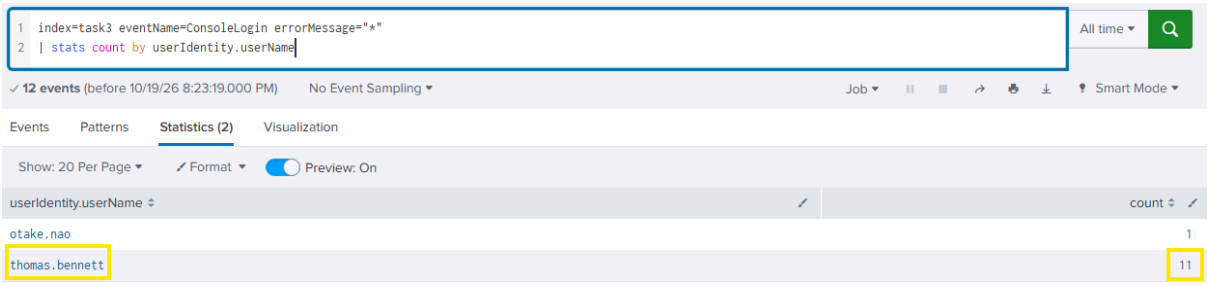
🔍 Investigação Prática no Splunk

Utilizei os logs do CloudTrail (`index=task3`) focando nos eventos `ConsoleLogin` para identificar tentativas anômalas e quebras na política de segurança (falta de MFA).

Passo 1: Identificar Falhas de Autenticação Filtrei as requisições que continham uma mensagem de erro (`errorMessage`), o que atesta a falha no acesso, agrupando os resultados pelo nome do usuário.

Splunk SPL
index=task3 eventName=ConsoleLogin errorMessage="*"
| stats count by userIdentity.userName

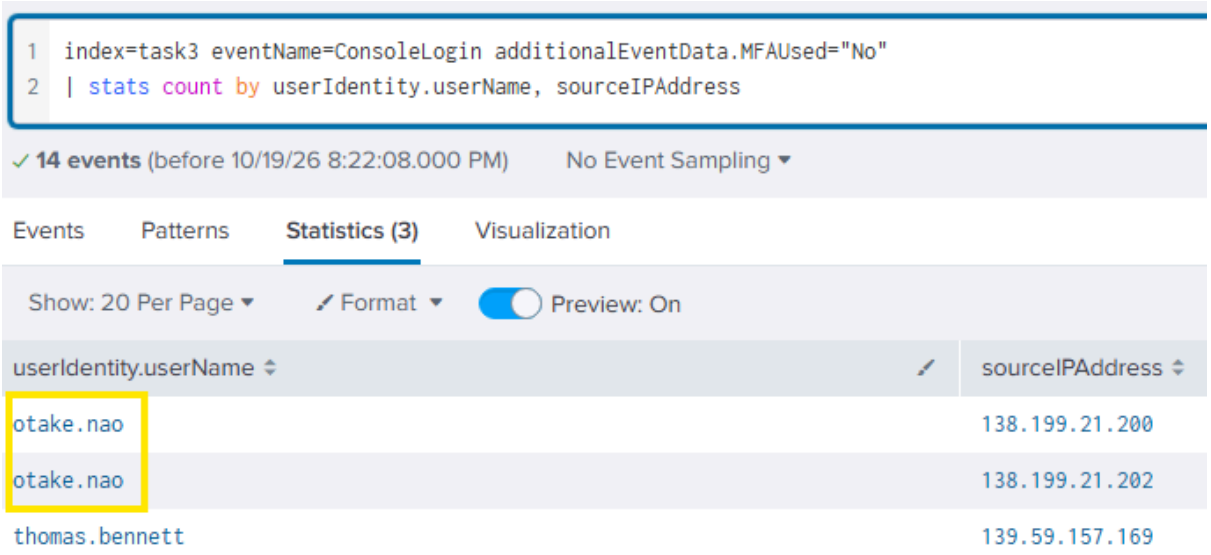
- **Pergunta:** Quantas vezes Thomas não conseguiu fazer login no console da AWS?
- **Resposta:** 11
- **Explicação:** Identificado contando a volumetria de eventos contendo o campo `errorMessage` (que registra a falha) associados ao usuário Thomas.



Passo 2: Detectar Autenticação sem MFA Para localizar brechas na verificação em duas etapas, filtrei logins bem-sucedidos (sem erro) onde o uso de MFA foi registrado como negativo.

Splunk SPL
index=task3 eventName=ConsoleLogin additionalEventData.MFAUsed="No"
| stats count by userIdentity.userName, sourceIPAddress

- **Pergunta:** Qual outro usuário fez login no console da AWS sem MFA?
- **Resposta:** otake.nao
- **Explicação:** Descoberto cruzando o evento `ConsoleLogin` com a condição exata do parâmetro `additionalEventData.MFAUsed="No"`, revelando qual conta acessou o ambiente sem a proteção extra.

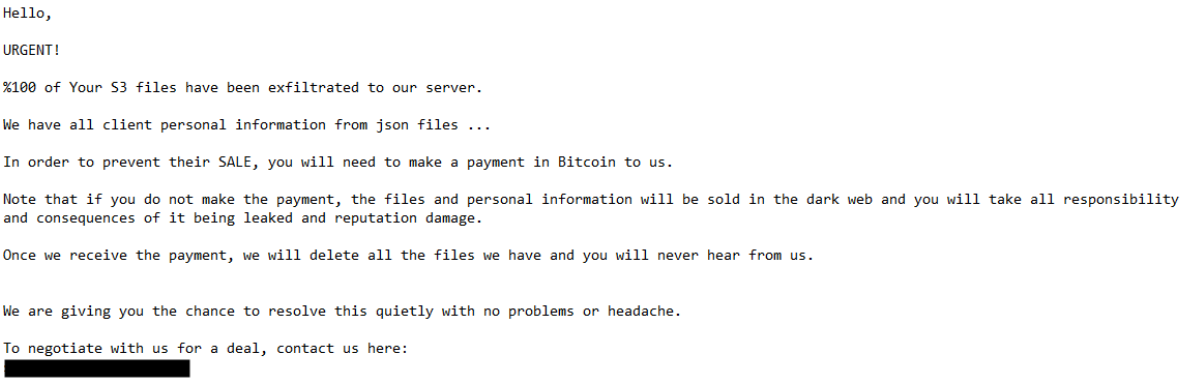


📌 **O que foi realizado**

- **O que foi feito:** Monitoramento e triagem de acessos ao Console da AWS por meio da análise de logs gerados pelo AWS CloudTrail no Splunk.
- **Conceitos aprendidos:** A estrutura do evento `ConsoleLogin` em JSON e a extração de parâmetros vitais para o SOC, como mensagens de erro e validações de verificação (MFA).
- **Importância:** O roubo de credenciais via *phishing* é uma das maiores causas de invasões em nuvem. Auditar continuamente o CloudTrail é a única forma de flagrar contas configuradas de forma insegura (sem MFA) ou detectar anomalias (como picos de erro) antes que os dados corporativos sejam exfiltrados.

Riscos e Monitoramento de Chaves de Acesso (Access Keys)

O uso programático da AWS, como a integração via Terraform ou scripts Python, exige o uso de chaves de acesso no lugar de senhas. Como essas chaves devem ser incluídas em cada chamada de API, desenvolvedores frequentemente cometem o erro crítico de armazená-las em texto claro no código-fonte, em bancos de dados ou em arquivos .env. O vazamento de chaves de acesso é a causa mais comum de violações graves na nuvem, levando a cenários como envio em massa de phishing via Amazon SES ou roubo de dados seguido de extorsão.



```
Hello,

URGENT!

%100 of Your S3 files have been exfiltrated to our server.

We have all client personal information from json files ...

In order to prevent their SALE, you will need to make a payment in Bitcoin to us.

Note that if you do not make the payment, the files and personal information will be sold in the dark web and you will take all responsibility
and consequences of it being leaked and reputation damage.

Once we receive the payment, we will delete all the files we have and you will never hear from us.


We are giving you the chance to resolve this quietly with no problems or headache.

To negotiate with us for a deal, contact us here:
[redacted]
```

Nota de resgate deixada pelo agente da ameaça (Unidade 42)

Monitoramento de Chaves de Acesso

Diferente do acesso via navegador, o acesso programático não registra o evento ConsoleLogin, pois as credenciais validam cada chamada de API individualmente. No console, uma ação costuma gerar uma cascata de eventos Get, List e Describe. Via CLI (Chaves de Acesso), o CloudTrail registra exclusivamente o comando que foi executado.

Para identificar se um ataque ocorreu via credenciais vazadas (Access Keys) em vez de uma sessão web sequestrada, podemos filtrar os logs no Splunk:

Splunk SPL
// Filtra apenas ações executadas via chaves de acesso
index=task4 userIdentity.accessKeyId=AKIA*

// Alternativa: filtra removendo sessões de console

index=task4 NOT userIdentity.sessionContext.attributes.creationDate=*

Pontos Focais na Investigação (Dicas de SOC)

A análise de scripts e automações pode ser confusa. Para diferenciar um comportamento benigno de um ataque:

- **Correlação de Propósito:** Um usuário de backup (svc-s3-backup) não deve, repentinamente, começar a provisionar instâncias EC2.
- **IP de Origem:** Contas de serviço (como um CI/CD do GitLab) não mudam de IP aleatoriamente. Um acesso via IP de outro provedor em nuvem (ex: DigitalOcean) é um alerta grave.
- **User-Agent (Cliente):** Mudanças abruptas do cliente utilizado (ex: de aws-sdk-java para aws-cli) podem indicar que um invasor externo importou as chaves vazadas para seu próprio terminal.

Investigação Prática no Splunk

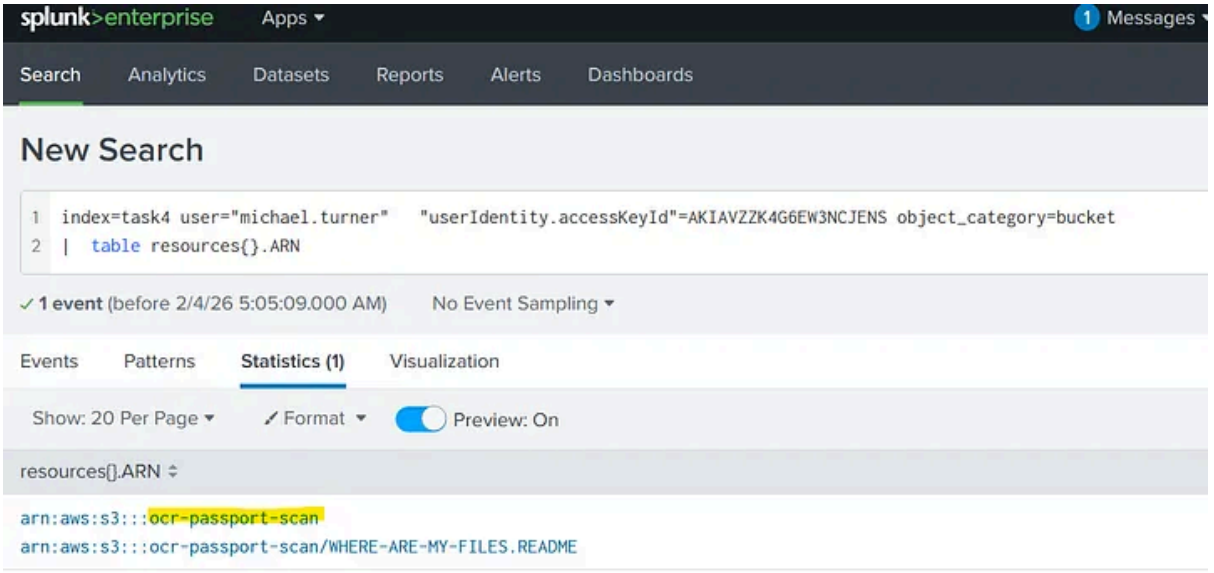
O foco foi rastrear um ataque de extorsão focado em um bucket S3 privado, originado através do comprometimento da chave de acesso do usuário "michael.turner".

Passo 1: Identificar a Chave de Acesso e o Alvo Filtrei as ações do usuário afetado para descobrir qual chave ele usava e qual recurso foi visado.

Splunk SPL
index=task4 userIdentity.userName="michael.turner"
| table userIdentity.accessKeyId, requestParameters.bucketName, eventName

- **Pergunta:** Qual ID de chave de acesso de Michael foi usado no ataque?
- **Resposta:** AKIAVZZK4G6EW3NCJENS
- **Explicação:** O campo userIdentity.accessKeyId exibiu a chave estática vazada que o atacante empregou.
- **Pergunta:** Qual é o nome do bucket S3 acessado pelos invasores?
- **Resposta:** ocr-passport-scan

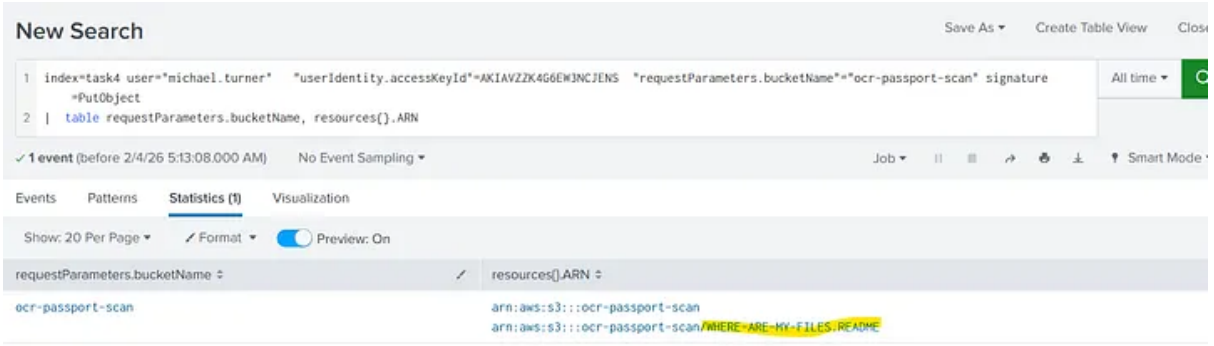
- **Explicação:** O alvo das requisições foi revelado através do campo requestParameters.bucketName.



Passo 2: Dimensionar a Exfiltração e o Resgate Para entender o tamanho do dano, filtrei exclusivamente os eventos de deleção no bucket S3 e as tentativas de gravação/upload.

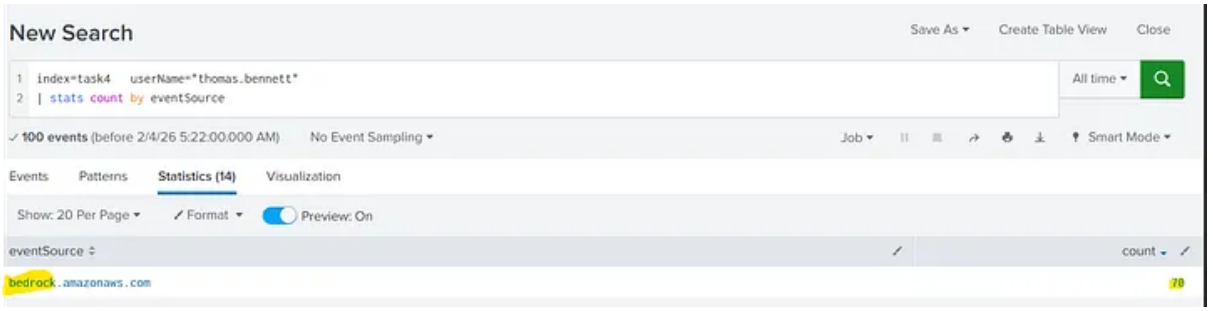
Splunk SPL
index=task4 requestParameters.bucketName="<NOME_DO_BUCKET>"
eventName="DeleteObject" OR eventName="PutObject"
| table eventName, requestParameters.key

- **Pergunta:** Quantos arquivos foram exfiltrados e excluídos pelo adversário?
- **Resposta:** 26
- **Explicação:** Contabilizado pela quantidade de eventos DeleteObject bem-sucedidos executados pelo atacante para apagar o conteúdo original.
- **Pergunta:** Qual arquivo foi carregado no bucket no final do ataque?
- **Resposta:** WHERE-ARE-MY-FILES.README
- **Explicação:** Revelado pelo evento final de PutObject, onde o invasor fez o upload da nota de resgate exigindo pagamento.



Passo 3: Auditoria do Console Aproveitando o ambiente, chequei rapidamente outros acessos não relacionados a chaves para confirmar atividades paralelas.

- **Pergunta:** Qual serviço da AWS foi mais utilizado pelo usuário que não usou chaves de acesso?
- **Resposta:** Amazon Bedrock
- **Explicação:** Encontrado filtrando eventos associados a contextos de sessão (sessionContext), que caracterizam o acesso visual (Console), e agrupando pelo campo eventSource.



O que foi realizado

- **O que foi feito:** Investigação de um ataque de extorsão na nuvem focado em roubo e destruição de dados no AWS S3, facilitado pelo comprometimento de Access Keys.
- **Conceitos aprendidos:** A diferenciação profunda nos logs do CloudTrail entre acessos de console (identificados por sessões) e acessos programáticos (userIdentity.accessKeyId). Técnicas de triagem de falsos positivos avaliando anomalias no propósito, IP e User-Agent da conta.
- **Importância:** Chaves expostas no GitHub ou arquivos de configuração lideram os incidentes de nuvem. No SOC (SIEM), correlacionar chamadas anômalas de API (como deleção em massa via CLI) é a última barreira de proteção para impedir que uma chave vazada resulte na perda definitiva dos dados críticos da organização.

Uso e Monitoramento de Funções IAM (IAM Roles) para Serviços

O uso principal das Funções IAM (IAM Roles) é permitir que serviços da AWS interajam entre si de forma segura. Em vez de utilizar chaves estáticas, uma carga de trabalho (como uma instância EC2 processando imagens em um bucket S3) assume uma função e opera com permissões temporárias.

Anatomia dos Logs de IAM Roles

Quando um serviço utiliza uma função IAM, os logs do CloudTrail refletem essa identidade temporária. Um evento típico (como `GetObject` no S3) apresentará as seguintes características:

- O tipo de identidade (`userIdentity.type`) é sempre registrado como `AssumedRole`.
- O ID da chave de acesso (`accessKeyId`) sempre começará com o prefixo **ASIA** (indicando credenciais temporárias do STS).
- O campo ARN da identidade termina com o **nome da sessão da função** (ex: `arn:aws:sts::123456789:assumed-role/NomeDaFuncao/NomeDaSessao`).

Por padrão, a AWS define o nome da sessão como o ID da instância (para EC2, ex: `i-123456789`) ou como o nome de usuário (para usuários IAM). No entanto, é possível personalizar esse nome (ex: `MyCustomSession`), o que oculta o autor original e exige investigação adicional.

O Evento `AssumeRole` e a Identificação do Paciente Zero

As funções IAM não operam sozinhas; sempre há um usuário ou serviço (o "paciente zero") que assume a função.

Quando um invasor ou usuário utiliza um nome de sessão personalizado, todas as ações subsequentes são registradas em nome da função, sem fazer referência ao usuário original. Para descobrir a verdadeira identidade por trás das ações, é estritamente necessário rastrear retrospectivamente o evento **`AssumeRole`** nos logs.

Esse evento registra a ação de solicitar a função e contém o campo crucial `userIdentity.userName` (o usuário real) e `requestParameters.roleSessionName` (o nome dado à sessão).

Abuso de IAM Roles e Acesso Cruzado (*Cross-Account*)

Abuso por Invasores: Se um adversário comprometer uma instância EC2 (por exemplo, via ataque de força bruta SSH), ele obtém controle total sobre ela e **herda automaticamente as mesmas permissões** da IAM Role anexada. Foi exatamente através desse abuso de IAM Roles que ocorreu o mega vazamento de dados da Capital One. Dicas de monitoramento em SOC:

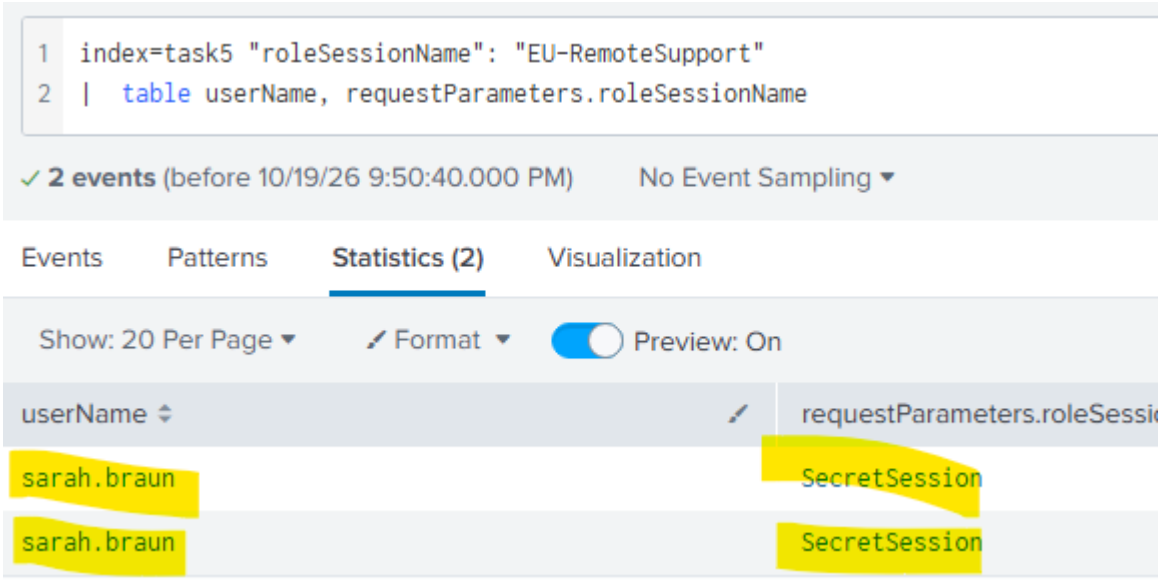
- Detectar mudanças repentinas no *user-agent* das chamadas.
- Caçar chamadas de API inéditas (*first-seen*), especialmente focadas em exfiltração de dados.
- Proteger ativamente as cargas de trabalho que utilizam as funções IAM.

Encadeamento de Funções (*Role Chaining*): O *Role Chaining* ocorre quando um usuário assume uma função em uma conta e, a partir dela, assume outra função em uma conta diferente. Para investigar incidentes nesses cenários complexos de acesso cruzado, o analista de segurança precisa "desencadear" a investigação, analisando os eventos `AssumeRole` um a um até encontrar o usuário original.

🔍 Investigação Prática no Splunk

Nesta etapa, analisei os logs do CloudTrail (`index=task5`) para rastrear a utilização legítima de funções por serviços AWS e "desmascarar" usuários escondidos atrás de sessões personalizadas.

- **Pergunta:** Qual ID de instância EC2 usou a função `UserAvatarsProcessor`?
- **Resposta:** i-0d2b8acdedc371589
- **Explicação:** Identificado ao analisar o final do campo ARN nos eventos disparados pela função, onde a AWS, por padrão, preenche o nome da sessão com o ID da instância operante.
- **Pergunta:** Alguém assumiu a função IAM `EU-RemoteSupport`. Como essa pessoa nomeou a sessão da função?
- **Resposta:** SecretSession
- **Explicação:** Descoberto filtrando as atividades executadas sob a função `EU-RemoteSupport` e observando o nome customizado registrado na extremidade do ARN.
- **Pergunta:** Qual usuário assumiu a função IAM da pergunta acima?
- **Resposta:** sarah.braun
- **Explicação:** Para encontrar essa resposta, realizei uma busca reversa pelo evento `AssumeRole` vinculado especificamente àquele nome de sessão personalizado, o que revelou a identidade raiz (usuário) que solicitou as credenciais temporárias do STS.



📌 O que foi realizado

- **O que foi feito:** Monitoramento e rastreamento de uso de funções (IAM Roles) na AWS através do Splunk, mapeando identidades ocultas atrás de sessões temporárias.
- **Conceitos aprendidos:** O funcionamento de credenciais temporárias (iniciadas em `ASIA`), a mecânica do evento `AssumeRole`, o conceito de *Role Chaining* (acesso cruzado) e a herança de privilégios através do comprometimento de instâncias.
- **Importância:** Funções IAM são constantemente abusadas por atacantes para escalar privilégios de forma invisível após o comprometimento de uma carga de

trabalho (como instâncias EC2). Analisar corretamente o AWS CloudTrail para conectar as ações executadas ao evento inicial de `AssumeRole` é a principal capacidade que um analista de SIEM precisa ter para identificar o verdadeiro "paciente zero" em ataques na nuvem.

Detecção de Alterações e Configurações Incorretas no IAM

Muitos ataques não começam no momento da exploração, mas sim meses ou anos antes, quando as condições para a invasão são formadas. Um exemplo comum: um administrador de TI cria uma chave de acesso `root` para testes e acidentalmente a expõe na internet. O cenário ideal para um SOC (*Security Operations Center*) não é detectar a invasão anos depois, mas sim monitorar e impedir a criação dessa chave indevida no momento exato em que ela ocorre.

Monitoramento de Configurações Incorretas no IAM

O monitoramento não deve focar apenas em sinais óbvios de comprometimento, mas também em falhas de configuração (*misconfigurations*). É fundamental auditar ações de administradores, como a atribuição de políticas excessivamente privilegiadas ou a criação de chaves desnecessárias.

Eventos Críticos de IAM para Monitoramento:

- **`PutUserPolicy`, `PutRolePolicy`, `PutGroupPolicy`:** Modificação de uma política existente. *Alerta:* Procure por divergências, como uma política "ReadOnlyAccess" sendo alterada para permitir a criação/exclusão de instâncias EC2.
- **`Attach[User|Group|Role]Policy`, `AddUserToGroup`:** Adição de uma identidade a um grupo ou atribuição de nova política. *Alerta:* Permissões como "AdministratorAccess" sendo concedidas a estagiários recém-contratados.
- **`DeleteVirtualMFADevice`, `DeactivateMFADevice`:** Desativação de MFA (próprio ou de terceiros). *Alerta:* Só é legítimo se for imediatamente seguido por um novo registro de MFA (ex: em caso de perda do celular).
- **`CreateAccessKey`, `UpdateAccessKey`:** Criação ou ativação de chaves de acesso IAM. *Alerta:* Chaves de acesso vazadas são o vetor de Acesso Inicial mais comum na AWS; confirme sempre com a TI se a criação era esperada.

Persistência e Acesso Inicial

Invasores frequentemente abusam do IAM tanto para acesso quanto para persistência. Um caso real analisado pela Permiso (grupo de ameaças *GUI-vil*) demonstrou atacantes acessando o ambiente via uma chave vazada e, em seguida, **criando uma segunda chave de acesso** como *backdoor*. Assim, se a TI descobrir e desativar a primeira chave, o invasor mantém o acesso através da segunda.

[Insira Imagem] *(Diagrama do ciclo de vida do ataque do grupo GUI-vil, ilustrando a criação de chaves de acesso para persistência)*

Dicas de Monitoramento:

- Monitore toda a atividade do usuário `root` e impeça tentativas de criação de chaves de acesso para essa conta.
- Foque nos eventos do IAM, especialmente no `CreateAccessKey`.
- Mantenha comunicação constante com a equipe de TI sobre configurações potencialmente vulneráveis.

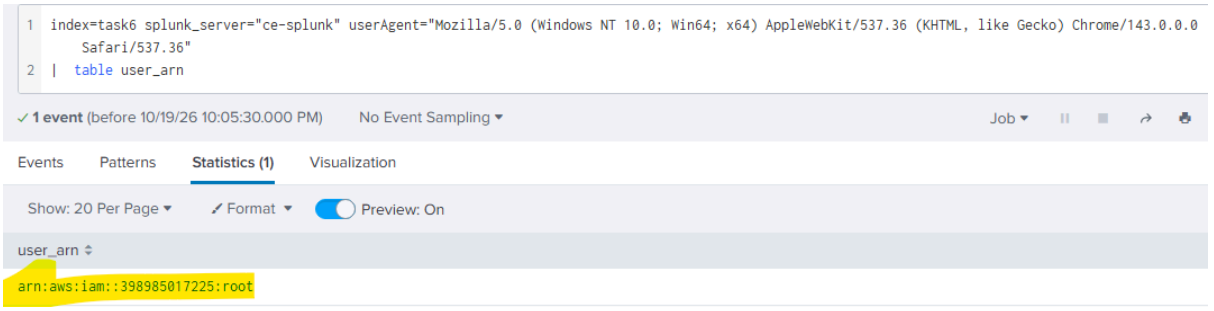
🔍 Investigação Prática no Splunk

Nesta etapa, auditei os logs (`index=task6`) de uma integração altamente insegura configurada entre o Splunk e o AWS CloudTrail para identificar as credenciais utilizadas e o momento de sua geração.

Passo 1: Identificar a Identidade da Integração Filtrei os eventos do CloudTrail para analisar as identidades (`userIdentity.arn`) atreladas à ingestão de logs do Splunk.

Splunk SPL
index=task6
| stats count by userIdentity.arn

- **Pergunta:** Sob qual ARN a integração do Splunk se autentica?
- **Resposta:** `arn:aws:iam::398985017225:root`
- **Explicação:** A extração do ARN revelou a identidade exata utilizada pelo serviço para se autenticar na AWS. *(Nota do SOC: Esta é uma configuração excepcionalmente insegura, pois o serviço utilizou um usuário IAM padrão em vez de assumir uma Função/Role apropriada).*

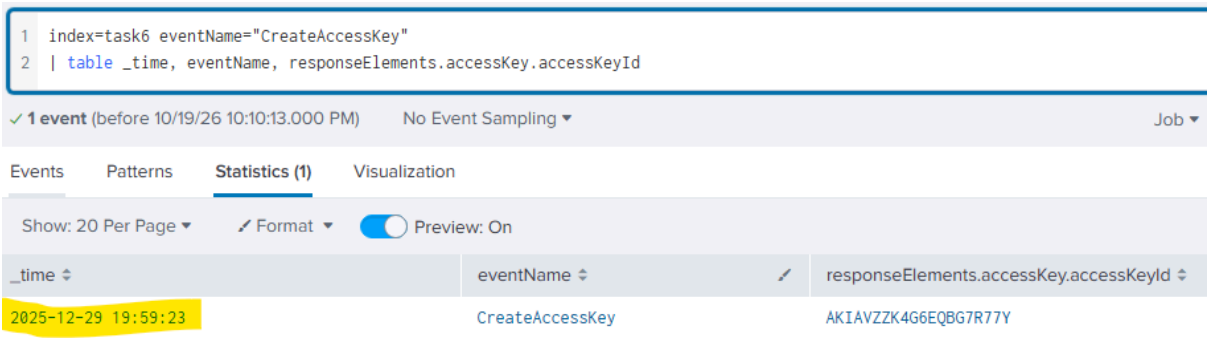


Passo 2: Rastrear a Criação da Chave de Acesso Para descobrir quando a chave superprivilegiada foi gerada, filtrei especificamente pelo evento de criação de chaves atrelado ao usuário da integração.

Splunk SPL
index=task6 eventName="CreateAccessKey"
| table _time, eventName, responseElements.accessKey.accessKeyId

- **Pergunta:** Quando a chave de acesso da integração excessivamente privilegiada foi criada?

- **Resposta:** [Inserir a Data e Hora exata]
- **Explicação:** O timestamp (`_time`) do evento `CreateAccessKey` detalhou o momento preciso em que a equipe (ou um atacante) gerou a credencial estática de longo prazo que viabilizou o acesso.



📌 O que foi realizado

- **O que foi feito:** Investigação proativa de eventos do IAM utilizando o Splunk para detectar falhas de configuração e exposição de credenciais em uma integração com o CloudTrail.
- **Conceitos aprendidos:** A importância de auditar ações de gerenciamento de políticas (`PutRolePolicy`, `AttachUserPolicy`) e de ciclo de vida de credenciais (`CreateAccessKey`, `DeactivateMFADevice`), além do entendimento de como atacantes abusam da criação de chaves para garantir Persistência.
- **Importância:** A defesa em nuvem não deve ser apenas reativa. Monitorar ativamente os logs de auditoria para detectar alterações de permissões (escalada de privilégios) e a criação de novas chaves de acesso (persistência) permite que o SOC neutralize as vulnerabilidades meses antes que um atacante consiga explorá-las.

Conclusão e Paralelo com Outras Nuvens

Esta documentação focou na fase de **Acesso Inicial** (*Initial Access*). Durante a investigação, ficou demonstrado como os invasores abusam de senhas do console de gerenciamento, chaves de acesso (*access keys*) e funções do IAM (*IAM roles*) para iniciar um ataque na AWS, e como essas ações podem ser detectadas e rastreadas utilizando os logs do CloudTrail.

A partir desse ponto de entrada inicial, um incidente pode ramificar-se em diversas direções prejudiciais para a organização, como extorsão em nuvem ou mineração de criptomoedas (*cryptomining*). As próximas investigações mergulharão mais fundo nos estágios subsequentes desses ataques.

Equivalência em Outros Provedores de Nuvem

Os conceitos de segurança de identidades da AWS possuem paralelos diretos (quase 1:1) nos outros principais provedores de nuvem. Compreender essas ideias de alto nível na AWS torna a aplicação da defesa muito mais fácil em ambientes *multicloud*:

- **Google Cloud (GCP):** As chaves de acesso e funções IAM mapeiam-se, respectivamente, para as **chaves de contas de serviço** (*service account keys*) e **contas de serviço** (*service accounts*).
- **Microsoft Azure:** Os mesmos recursos correspondem às **credenciais de entidade de serviço** (*service principal credentials*) e **identidades gerenciadas** (*managed identities*).

📌 O que foi realizado

- **O que foi feito:** Conclusão do módulo de Acesso Inicial na nuvem, resumindo os vetores de entrada (senhas do console, *access keys* e *roles*) e mapeando as tecnologias correspondentes em outros provedores.
- **Conceitos aprendidos:** A equivalência direta das estruturas de identidade e acesso entre AWS, Google Cloud (*service accounts*) e Microsoft Azure (*managed identities*), bem como os possíveis desdobramentos de um ataque (extorsão e *cryptomining*).
- **Importância:** Compreender a mecânica do Acesso Inicial na AWS não limita o analista a apenas uma plataforma. A arquitetura de chaves programáticas e identidades assumidas é um padrão da indústria, o que significa que a mesma lógica de caça a ameaças e monitoramento de logs pode ser aplicada para defender qualquer ambiente corporativo em nuvem.